JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ZipFile

java.lang.Object
    java.util.zip.ZipFile

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`JarFile`

```
public class ZipFile
extends Object
implements Closeable
```

This class is used to read entries from a ZIP file.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

API Note:

To release resources used by this `ZipFile`, the `close()` method should be called explicitly or by try-with-resources. Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses that override `Object.finalize()` in order to perform cleanup should be modified to use alternative cleanup mechanisms such as `Cleaner` and remove the overriding `finalize` method.

Since:

1.1

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	**`CENATT`**	Central directory (CEN) header internal file attributes field offset.
`static final int`	**`CENATX`**	Central directory (CEN) header external file attributes field offset.
`static final int`	**`CENCOM`**	Central directory (CEN) header comment length field offset.

static final int	**CENCRC**	Central directory (CEN) header uncompressed file crc-32 value field offset.
static final int	**CENDSK**	Central directory (CEN) header disk number start field offset.
static final int	**CENEXT**	Central directory (CEN) header extra field length field offset.
static final int	**CENFLG**	Central directory (CEN) header encrypt, decrypt flags field offset.
static final int	**CENHDR**	Central directory (CEN) header size in bytes (including signature).
static final int	**CENHOW**	Central directory (CEN) header compression method field offset.
static final int	**CENLEN**	Central directory (CEN) header uncompressed size field offset.
static final int	**CENNAM**	Central directory (CEN) header filename length field offset.
static final int	**CENOFF**	Central directory (CEN) header LOC header offset field offset.
static final long	**CENSIG**	Central directory (CEN) header signature.
static final int	**CENSIZ**	Central directory (CEN) header compressed size field offset.
static final int	**CENTIM**	Central directory (CEN) header modification time field offset.

static final int	**CENVEM**	Central directory (CEN) header version made by field offset.
static final int	**CENVER**	Central directory (CEN) header version needed to extract field offset.
static final int	**ENDCOM**	End of central directory (END) header ZIP file comment length field offset.
static final int	**ENDHDR**	End of central directory (END) header size in bytes (including signature).
static final int	**ENDOFF**	End of central directory (END) header offset for the first CEN header field offset.
static final long	**ENDSIG**	End of central directory (END) header signature.
static final int	**ENDSIZ**	End of central directory (END) header central directory size in bytes field offset.
static final int	**ENDSUB**	End of central directory (END) header number of entries on this disk field offset.
static final int	**ENDTOT**	End of central directory (END) header total number of entries field offset.
static final int	**EXTCRC**	Extra local (EXT) header uncompressed file crc-32 value field offset.
static final int	**EXTHDR**	

		Extra local (EXT) header size in bytes (including signature).
static final int	**EXTLEN**	Extra local (EXT) header uncompressed size field offset.
static final long	**EXTSIG**	Extra local (EXT) header signature.
static final int	**EXTSIZ**	Extra local (EXT) header compressed size field offset.
static final int	**LOCCRC**	Local file (LOC) header uncompressed file crc-32 value field offset.
static final int	**LOCEXT**	Local file (LOC) header extra field length field offset.
static final int	**LOCFLG**	Local file (LOC) header general purpose bit flag field offset.
static final int	**LOCHDR**	Local file (LOC) header size in bytes (including signature).
static final int	**LOCHOW**	Local file (LOC) header compression method field offset.
static final int	**LOCLEN**	Local file (LOC) header uncompressed size field offset.
static final int	**LOCNAM**	Local file (LOC) header filename length field offset.
static final long	**LOCSIG**	Local file (LOC) header signature.
static final int	**LOCSIZ**	

		Local file (LOC) header compressed size field offset.
static final int	**LOCTIM**	Local file (LOC) header modification time field offset.
static final int	**LOCVER**	Local file (LOC) header version needed to extract field offset.
static final int	**OPEN_DELETE**	Mode flag to open a ZIP file and mark it for deletion.
static final int	**OPEN_READ**	Mode flag to open a ZIP file for reading.

Constructor Summary

Constructors

Constructor	Description
ZipFile(File file)	Opens a ZIP file for reading given the specified File object.
ZipFile(File file, int mode)	Opens a new ZipFile to read from the specified File object in the specified mode.
ZipFile(File file, int mode, Charset charset)	Opens a new ZipFile to read from the specified File object in the specified mode.
ZipFile(File file, Charset charset)	Opens a ZIP file for reading given the specified File object.
ZipFile(String name)	Opens a ZIP file for reading.
ZipFile(String name, Charset charset)	Opens a ZIP file for reading.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
void	**close**()	Closes the ZIP file.
Enumeration<? extends ZipEntry>	**entries**()	Returns an enumeration of the ZIP file entries.
String	**getComment**()	Returns the ZIP file comment.
ZipEntry	**getEntry**(String name)	Returns the ZIP file entry for the specified name, or null if not found.
InputStream	**getInputStream**(ZipEntry entry)	Returns an input stream for reading the contents of the specified ZIP file entry.
String	**getName**()	Returns the path name of the ZIP file.
int	**size**()	Returns the number of entries in the ZIP file.
Stream<? extends ZipEntry>	**stream**()	Returns an ordered `Stream` over the ZIP file entries.
String	**toString**()	Returns a string identifying this `ZipFile`, for debugging.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Field Details

OPEN_READ

`public static final int OPEN_READ`

Mode flag to open a ZIP file for reading.

See Also:

Constant Field Values

OPEN_DELETE

`public static final int OPEN_DELETE`

Mode flag to open a ZIP file and mark it for deletion. The file will be deleted some time between the moment that it is opened and the moment that it is closed, but its contents will remain accessible via the `ZipFile` object until either the close method is invoked or the virtual machine exits.

See Also:

Constant Field Values

LOCSIG

`static final long LOCSIG`

Local file (LOC) header signature.

See Also:

Constant Field Values

EXTSIG

`static final long EXTSIG`

Extra local (EXT) header signature.

See Also:

Constant Field Values

CENSIG

`static final long CENSIG`

Central directory (CEN) header signature.

See Also:

Constant Field Values

ENDSIG

```
static final long ENDSIG
```

End of central directory (END) header signature.

See Also:

Constant Field Values

LOCHDR

```
static final int LOCHDR
```

Local file (LOC) header size in bytes (including signature).

See Also:

Constant Field Values

EXTHDR

```
static final int EXTHDR
```

Extra local (EXT) header size in bytes (including signature).

See Also:

Constant Field Values

CENHDR

```
static final int CENHDR
```

Central directory (CEN) header size in bytes (including signature).

See Also:

Constant Field Values

ENDHDR

```
static final int ENDHDR
```

End of central directory (END) header size in bytes (including signature).

See Also:

Constant Field Values

LOCVER

```
static final int LOCVER
```

Local file (LOC) header version needed to extract field offset.

See Also:

Constant Field Values

LOCFLG

```
static final int LOCFLG
```

Local file (LOC) header general purpose bit flag field offset.

See Also:

Constant Field Values

LOCHOW

```
static final int LOCHOW
```

Local file (LOC) header compression method field offset.

See Also:

Constant Field Values

LOCTIM

```
static final int LOCTIM
```

Local file (LOC) header modification time field offset.

See Also:

Constant Field Values

LOCCRC

```
static final int LOCCRC
```

Local file (LOC) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

LOCSIZ

```
static final int LOCSIZ
```

Local file (LOC) header compressed size field offset.

See Also:

Constant Field Values

LOCLEN

`static final int LOCLEN`

Local file (LOC) header uncompressed size field offset.

See Also:

Constant Field Values

LOCNAM

`static final int LOCNAM`

Local file (LOC) header filename length field offset.

See Also:

Constant Field Values

LOCEXT

`static final int LOCEXT`

Local file (LOC) header extra field length field offset.

See Also:

Constant Field Values

EXTCRC

`static final int EXTCRC`

Extra local (EXT) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

EXTSIZ

`static final int EXTSIZ`

Extra local (EXT) header compressed size field offset.

See Also:

Constant Field Values

EXTLEN

```
static final int EXTLEN
```

Extra local (EXT) header uncompressed size field offset.

See Also:

Constant Field Values

CENVEM

```
static final int CENVEM
```

Central directory (CEN) header version made by field offset.

See Also:

Constant Field Values

CENVER

```
static final int CENVER
```

Central directory (CEN) header version needed to extract field offset.

See Also:

Constant Field Values

CENFLG

```
static final int CENFLG
```

Central directory (CEN) header encrypt, decrypt flags field offset.

See Also:

Constant Field Values

CENHOW

```
static final int CENHOW
```

Central directory (CEN) header compression method field offset.

See Also:

Constant Field Values

CENTIM

```
static final int CENTIM
```

Central directory (CEN) header modification time field offset.

See Also:

Constant Field Values

CENCRC

`static final int CENCRC`

Central directory (CEN) header uncompressed file crc-32 value field offset.

See Also:

Constant Field Values

CENSIZ

`static final int CENSIZ`

Central directory (CEN) header compressed size field offset.

See Also:

Constant Field Values

CENLEN

`static final int CENLEN`

Central directory (CEN) header uncompressed size field offset.

See Also:

Constant Field Values

CENNAM

`static final int CENNAM`

Central directory (CEN) header filename length field offset.

See Also:

Constant Field Values

CENEXT

`static final int CENEXT`

Central directory (CEN) header extra field length field offset.

See Also:

Constant Field Values

CENCOM

```
static final int CENCOM
```

Central directory (CEN) header comment length field offset.

See Also:

Constant Field Values

CENDSK

```
static final int CENDSK
```

Central directory (CEN) header disk number start field offset.

See Also:

Constant Field Values

CENATT

```
static final int CENATT
```

Central directory (CEN) header internal file attributes field offset.

See Also:

Constant Field Values

CENATX

```
static final int CENATX
```

Central directory (CEN) header external file attributes field offset.

See Also:

Constant Field Values

CENOFF

```
static final int CENOFF
```

Central directory (CEN) header LOC header offset field offset.

See Also:

Constant Field Values

ENDSUB

```
static final int ENDSUB
```

End of central directory (END) header number of entries on this disk field offset.

See Also:

Constant Field Values

ENDTOT

```
static final int ENDTOT
```

End of central directory (END) header total number of entries field offset.

See Also:

Constant Field Values

ENDSIZ

```
static final int ENDSIZ
```

End of central directory (END) header central directory size in bytes field offset.

See Also:

Constant Field Values

ENDOFF

```
static final int ENDOFF
```

End of central directory (END) header offset for the first CEN header field offset.

See Also:

Constant Field Values

ENDCOM

```
static final int ENDCOM
```

End of central directory (END) header ZIP file comment length field offset.

See Also:

Constant Field Values

Constructor Details

ZipFile

```
public ZipFile(String name)
        throws IOException
```

Opens a ZIP file for reading.

The UTF-8 `charset` is used to decode the entry names and comments.

Parameters:

`name` - the name of the ZIP file

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

ZipFile

```
public ZipFile(File file,
               int mode)
        throws IOException
```

Opens a new `ZipFile` to read from the specified `File` object in the specified mode. The mode argument must be either `OPEN_READ` or `OPEN_READ | OPEN_DELETE`.

The UTF-8 `charset` is used to decode the entry names and comments

Parameters:

`file` - the ZIP file to be opened for reading

`mode` - the mode in which the file is to be opened

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

`IllegalArgumentException` - if the `mode` argument is invalid

Since:

1.3

ZipFile

```
public ZipFile(File file)
        throws ZipException,
               IOException
```

Opens a ZIP file for reading given the specified File object.

The UTF-8 `charset` is used to decode the entry names and comments.

Parameters:

`file` - the ZIP file to be opened for reading

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

ZipFile

```
public ZipFile(File file,
               int mode,
               Charset charset)
        throws IOException
```

Opens a new `ZipFile` to read from the specified `File` object in the specified mode. The mode argument must be either `OPEN_READ` or `OPEN_READ | OPEN_DELETE`.

Parameters:

`file` - the ZIP file to be opened for reading

`mode` - the mode in which the file is to be opened

`charset` - the charset to be used to decode the ZIP entry name and comment that are not encoded by using UTF-8 encoding (indicated by entry's general purpose flag).

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

`IllegalArgumentException` - if the `mode` argument is invalid

Since:

1.7

ZipFile

```
public ZipFile(String name,
               Charset charset)
        throws IOException
```

Opens a ZIP file for reading.

Parameters:

`name` - the name of the ZIP file

`charset` - the charset to be used to decode the ZIP entry name and comment that are not encoded by using UTF-8 encoding (indicated by entry's general purpose flag).

Throws:

`ZipException` - if a ZIP format error has occurred

IOException - if an I/O error has occurred

Since:

1.7

ZipFile

```
public ZipFile(File file,
               Charset charset)
        throws IOException
```

Opens a ZIP file for reading given the specified File object.

Parameters:

file - the ZIP file to be opened for reading

charset - The charset to be used to decode the ZIP entry name and comment (ignored if the language encoding bit of the ZIP entry's general purpose bit flag is set).

Throws:

ZipException - if a ZIP format error has occurred

IOException - if an I/O error has occurred

Since:

1.7

Method Details

getComment

```
public String getComment()
```

Returns the ZIP file comment. If a comment does not exist or an error is encountered decoding the comment using the charset specified when opening the ZIP file, then null is returned.

Returns:

the comment string for the ZIP file, or null if none

Throws:

IllegalStateException - if the ZIP file has been closed

Since:

1.7

getEntry

```
public ZipEntry getEntry(String name)
```

Returns the ZIP file entry for the specified name, or null if not found.

Parameters:

`name` - the name of the entry

Returns:

the ZIP file entry, or null if not found

Throws:

`IllegalStateException` - if the ZIP file has been closed

getInputStream

```
public InputStream getInputStream(ZipEntry entry)
                           throws IOException
```

Returns an input stream for reading the contents of the specified ZIP file entry.

Closing this ZIP file will, in turn, close all input streams that have been returned by invocations of this method.

API Note:

The `InputStream` returned by this method can wrap an `InflaterInputStream`, whose `read(byte[], int, int)` method can modify any element of the output buffer.

Parameters:

`entry` - the ZIP file entry

Returns:

the input stream for reading the contents of the specified ZIP file entry or null if the ZIP file entry does not exist within the ZIP file.

Throws:

`ZipException` - if a ZIP format error has occurred

`IOException` - if an I/O error has occurred

`IllegalStateException` - if the ZIP file has been closed

getName

```
public String getName()
```

Returns the path name of the ZIP file.

Returns:

the path name of the ZIP file

toString

```
public String toString()
```

Returns a string identifying this `ZipFile`, for debugging.

Overrides:

`toString` in class `Object`

Returns:

a string identifying this `ZipFile`, for debugging

entries

```
public Enumeration<? extends ZipEntry> entries()
```

Returns an enumeration of the ZIP file entries.

Returns:

an enumeration of the ZIP file entries

Throws:

`IllegalStateException` - if the ZIP file has been closed

stream

```
public Stream<? extends ZipEntry> stream()
```

Returns an ordered `Stream` over the ZIP file entries. Entries appear in the `Stream` in the order they appear in the central directory of the ZIP file.

Returns:

an ordered `Stream` of entries in this ZIP file

Throws:

`IllegalStateException` - if the ZIP file has been closed

Since:

1.8

size

```
public int size()
```

Returns the number of entries in the ZIP file.

Returns:

the number of entries in the ZIP file

Throws:

`IllegalStateException` - if the ZIP file has been closed

close

```
public void close()
           throws IOException
```

Closes the ZIP file.

Closing this ZIP file will close all of the input streams previously returned by invocations of the `getInputStream` method.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Throws:

`IOException` - if an I/O error has occurred

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.